

Artificial Intelligence (AI)

Principles -

- (i) Thinking humanly
- (ii) Acting humanly
- (iii) Thinking rationally
- (iv) Acting rationally

Applications -

- (i) Health Care - disease diagnosis
- (ii) Education - intelligent tutoring systems
- (iii) Industry - automation and robotics
- (iv) Agriculture
- (v) Retail & E-commerce
- (vi) Media
- (vii) Logistics

AI Problem -

- (i) Lack of generalization
- (ii) Data Privacy & Security
- (iii) Bias and Fairness issues
- (iv) High computational costs
- (v) Explainability and transparency
- (vi) Ethical and legal concerns

Intelligent Agent -

An entity that perceives its environment and acts upon it.
eg. Robot, software bot, virtual assistants, data agents, etc

Key characteristics:

- Autonomy
- Reactivity
- Pro-activeness
- Social ability

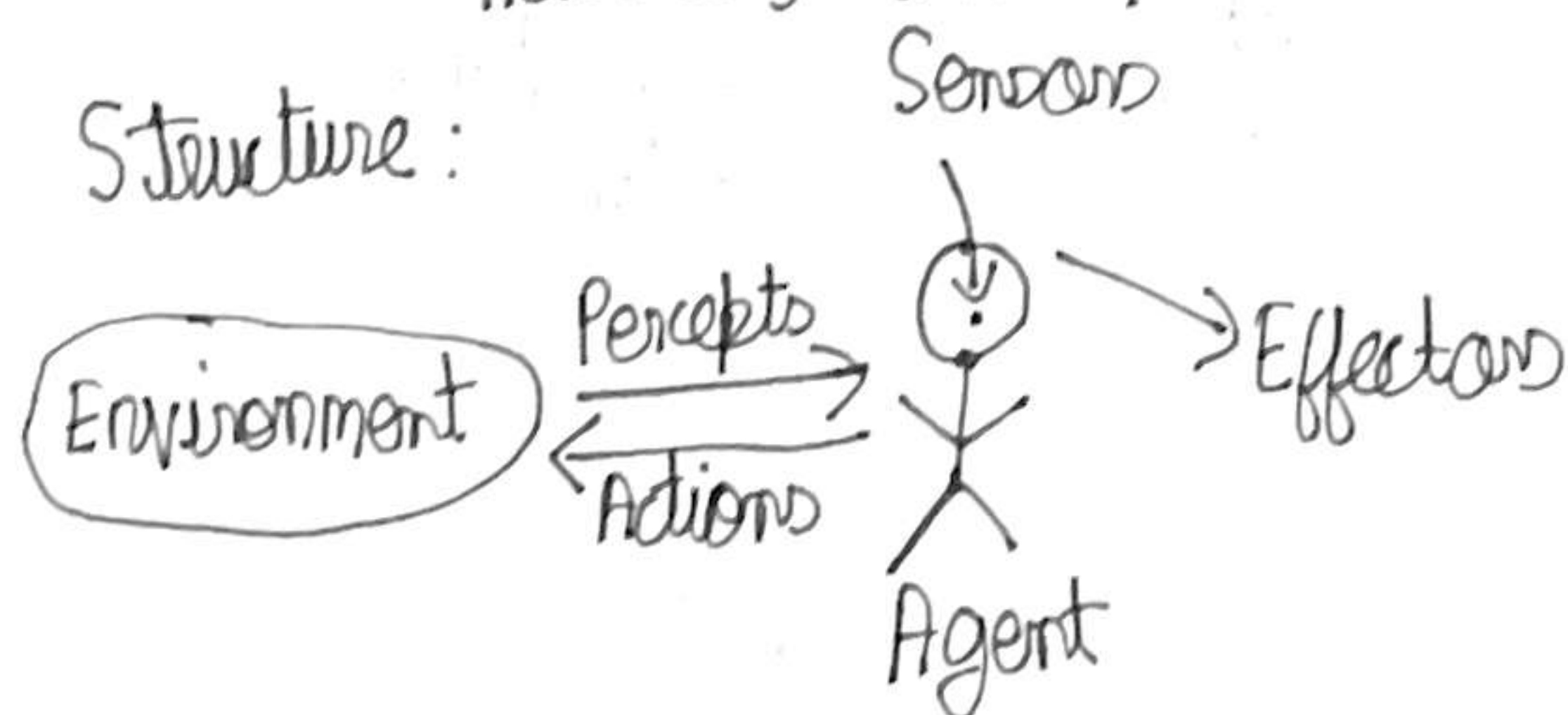
What agents do?

- Perceive the environment
- Reason and plan
- Take action
- Adapt and learn
- Use tools

Components:

- Sensors = to perceive the environment
- Actuators = to act in the environment
- Processor (Agent Function) = decides what action to take
- eg = Robotic vacuum cleaner
 - Sensors = infrared, bump sensors
 - Actuators = wheels, suction motor

Structure:



Environment:

External world in which an agent operates

Properties:

- observable vs partially observable
- deterministic vs stochastic
- episodic vs sequential
- static vs dynamic
- discrete vs continuous

PEAS Framework —

P = Performance Measure: Success criteria
 E = Environment: External conditions
 A = Actuators: Tools to act
 S = Sensors: Tools to perceive

Concepts of Rationality —

Types:

Bounded rationality
 Perfect rationality

Types of Agents —

- | | |
|--------------------------------|--------------------------------|
| (i) Simple reflex agents | (v) Learning agents |
| (ii) Model based reflex agents | (vi) Multi Agent Systems (MAS) |
| (iii) Goal based agents | (vii) Hierarchical agents |
| (iv) Utility based agents | |

Agent Type	Strength	Limitations	Best for	Examples
1) Simple reflex	Instant reaction based on fixed rules	No memory or learning; fails in dynamic environments	Fully observable, stable and simple environments	Traffic Light Timers
2) Model based	Handles partial observability with internal state	More computational demand; depend on model accuracy	Dynamic or partially observable environments	Robot vacuum cleaners
3) Goal Based	Plans ahead to achieve specific objectives	Needs clear goals & planning algorithms	Strategic tasks with defined goals	Logistics route planning
4) Utility Based	Balances multiple factors for best outcome	Requires complex utility functions	Multi-criteria decision making	Financial portfolio management
5) Learning Agent	Improves over time via experience	Needs data & training time	Dynamic environments with changing conditions	AI chatbots
6) Multi Agent System (MAS)	Distributed problem solving with cooperation or competition	Complex interactions; unpredictable behavior	Decentralized, multi entity systems	Smart traffic control
7) Hierarchical Agent	Breaks complex tasks into levels for efficiency	Requires well defined interfaces between layers	Large scale multi level operations	Drone Delivery

Agents work by:

- (i) Perceiving the environment
- (ii) Defining the problem
- (iii) Exploring different possibilities
- (iv) Evaluating and deciding
- (v) Learning and adapting

Key components of Problem Formulation:

- (i) Initial state
- (ii) Action
- (iii) Transition
- (iv) Goal Test
- (v) Cost function

State Space Representation -

State space is the set of all possible states from the initial state to the goal state through valid operations

Components:

- (i) Initial state
- (ii) Goal state
- (iii) Operators
- (iv) State space graph
- (v) Path cost

State Space Search Strategies -

- (i) Informed
 - A* algorithm
 - Best First Search
- (ii) Uninformed
 - DFS
 - BFS
 - Depth Limited
 - Uniform cost
 - Iterative deepening DFS
- (iii) Adversarial
 - Min max
 - Alpha-Beta Pruning
- (iv) Local Search
 - Hill Climbing
 - Genetic algorithm

Search techniques

Blind
(Uninformed Search)

Heuristic
(Informed Search)

Depth first
Search
(DFS)

Breadth first
Search
(BFS)

Hill
Climbing
Search

A*
Search

Best First
Search

Greedy
Search

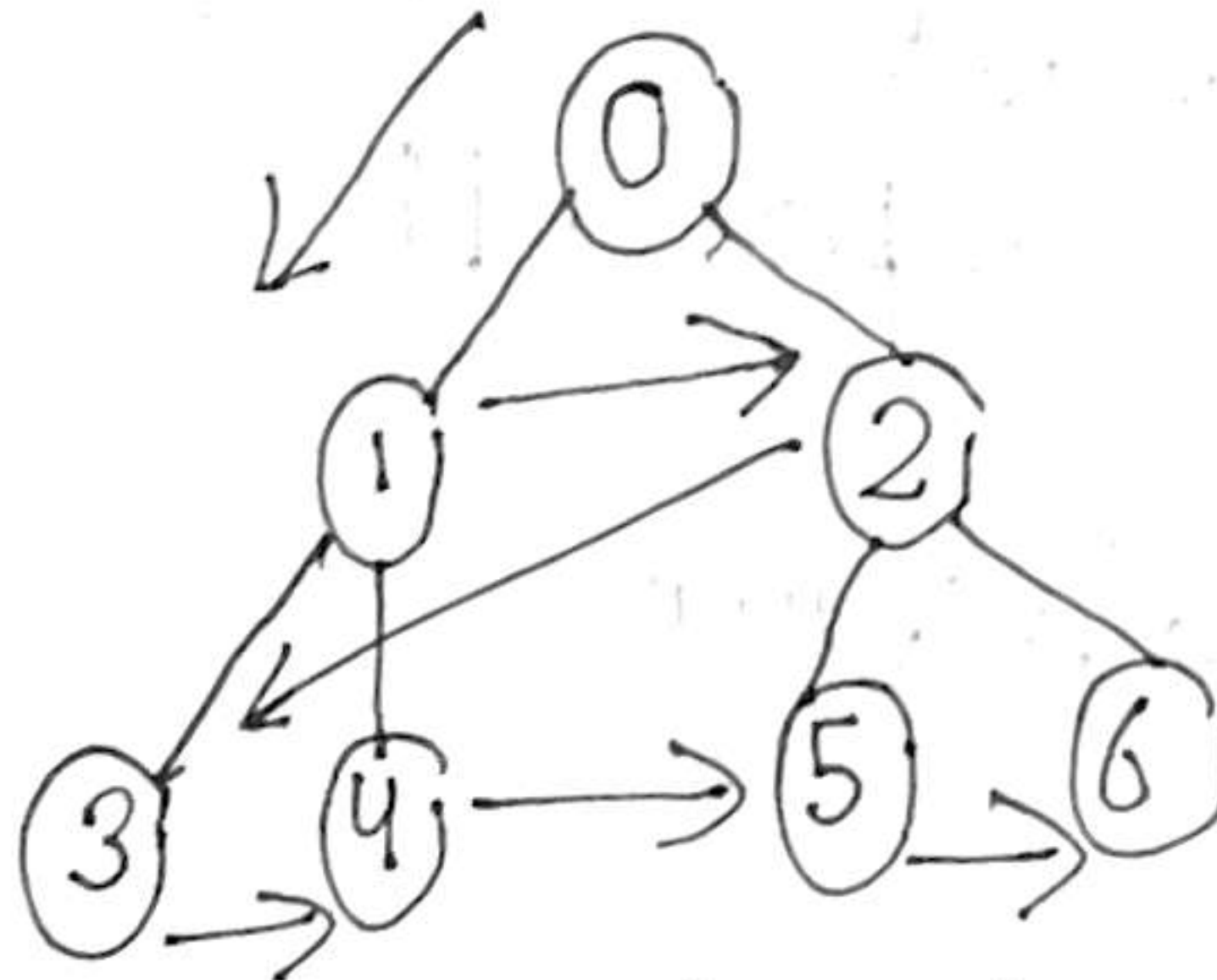
Other Blind search
strategies are:

- Depth limited search (extended DFS)
- Iterative deepening search (extended DFS)
- Uniform cost search
- Bi directional search

Uninformed Search —
Search without using extra problem knowledge

Types:

- BFS
- DFS



BFS (queue)

- explores all nodes at current depth before going deeper (shallowest node)
- Guaranteed solution
- Finds shortest path



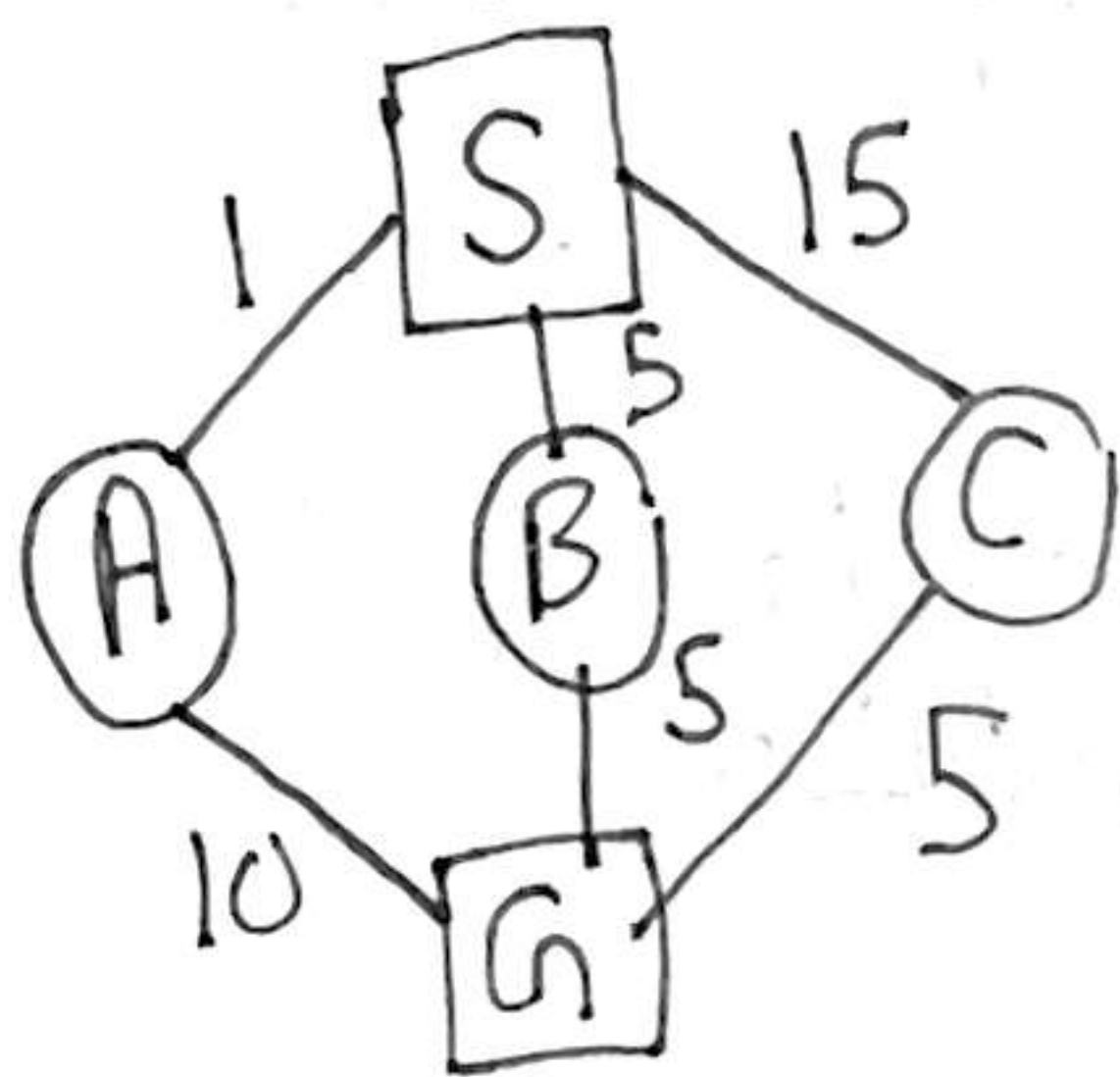
DFS (stack)

- explores deepest node first
- does not guarantee optimal solution
- efficient when solution depth is unknown
- Can get trapped in loops
- Requires very little memory

Aspect	BFS	DFS
Advantages	- Guaranteed solution - Finds shortest path	- Low memory - First in deep problems
Use Case	- Shortest path problems - Robot navigation - Puzzle problems	- Game tree search - Backtracking problems - Constraint solving
Disadvantages	- Requires very high memory	- Does not guarantee optimal solution

Uniform cost search -

- Modified breadth first, expands only the node with the least cost (using path $g(n)$).
- Breadth first = uniform cost search with $g(n) = \text{depth}(n)$



SAG is not a solution
 As SBG has less cost
 $11 > 10$

Informed Search Strategies -

- Uses additional information called heuristics
- Unlike uninformed search, it makes intelligent decisions about which node to explore next
- Heuristic function $h(n)$ = estimated cost to goal
- eg. distance in GPS, tiles misplaced puzzle, evaluation function in games

- AI algorithms select nodes using: $f(n) = g(n) + h(n)$
 where, $g(n)$ = actual cost from start
 $h(n)$ = estimated cost to goal
 $f(n)$ = total estimated cost

Heuristic functions -

- Simple heuristics for 8 puzzle:

└ $h_1(n)$ = no. of misplaced tiles

└ $h_2(n)$ = total Manhattan distance

- $h_1(s) = 8$

$h_2(s) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$

- dominance:

└ $h_2(n) > h_1(n)$ for all n (both admissible)

└ h_2 is better for search (closer to perfect)

└ less nodes need to be expanded

7	2	4
5		6
8	3	1

Start state

	1	2
3	4	5
6	7	8

Goal state

Greedy Best first search algorithm -

- Attempts to find the most promising path from a given starting point to a goal

- It prioritizes paths that appear to be the most promising

- Process is repeated until the goal is reached

- Performance measure:

└ Completeness: It is incomplete even in finite state space

└ Optimality: does not provide an optimal solution

└ Combines the advantages of BFS & DFS

└ Advantages:

└ Fast

└ Simple

└ Disadvantages:

└ Does not guarantee to reach the goal state

└ May cover a long distance in some cases

└ Does not give an optimized solution (greedy approach)

Hill Climbing -

- A local search algorithm
- Starts from an initial state & moves to best neighboring state
- Uses greedy approach
- Stops when no better neighbor exists
- Variants of generate and test
- Always moves towards better cost/value
- Optimizes the objective (cost) function

- Types:
 - Simple = chooses first better neighbor
 - Steepest ascent = chooses better best among all neighbors
 - Stochastic = random neighbor selection

- Problems:
 - May ~~search~~ fail to reach global maximum
 - Gets stuck due to search space structure

Local maximum:

- No neighbor has better value
- Algorithms stop early

Plateau:

- All neighbors have same value
- No clear direction to move

Ridge:

- Narrow peak, hard to climb
- Greedy moves fail

- Overcoming Problems:

- Local maximum: backtracking
- Plateau: random jump
- Ridge: multi directional moves

- Algorithm:

- Start with initial state
- Evaluate current state
- Generate a neighbor
- If better $\rightarrow$ move to it
- Repeat until no improvement
- Stop

#A* Search Algorithm-

- An informed (heuristic) search algorithm
- Combines path cost and heuristic estimate
- Evaluation function: $f(n) = g(n) + h(n)$

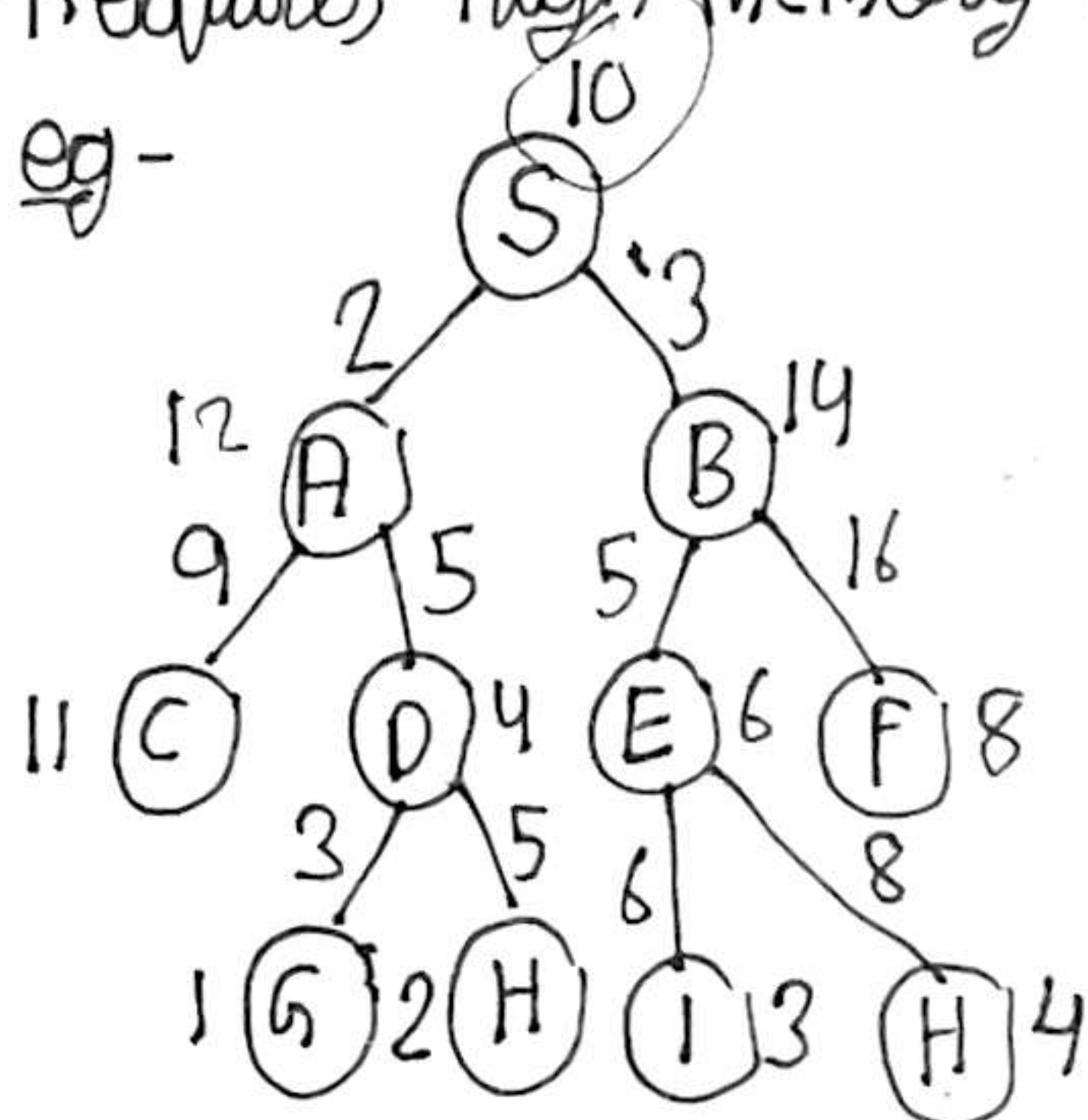
where, $f(n)$ = estimated total cost of solution

$h(n)$ = estimated cost from n to goal

$g(n)$ = cost from start node to node n

- Optimal and complete (with admissible heuristic)
- Requires high memory and good heuristic

- eg -



- Start from root node S
- In order to reach G, calculate $f(n)$ value of node S, A and B
 $f(n) = g(n) + h(n)$

- Calculation of $f(n)$ for node S:

$$f(S) = (\text{distance from node S to S}) + h(S) \\ = 0 + 10 = 10$$

- Calculation of $f(n)$ for node A:

$$f(A) = (\text{distance from S to A}) + h(A) \\ = 2 + 12 \\ = 14$$

- Calculation of $f(n)$ for node B:

$$f(B) = (\text{distance from node S to B}) + h(B) \\ = 3 + 14 \\ = 17$$

Since node A has the lowest $f(n)$ value, node A will be explored to the next nodes C & D

& after calculating, the sequence we get is $S \rightarrow A \rightarrow D \rightarrow G$

Performance measure of A* search:

- [Completeness: the star (*) in A* search guarantees to reach the goal node
- [Optimality: an ~~underest~~ underestimated cost will always give an optimal solution

Advantages

- 1) Finds optimal path (if heuristic is admissible)
- 2) Complete - always finds a solution
- 3) Faster than BFS/DFS with good heuristic
- 4) Uses knowledge to guide search

Disadvantages

- 1) Uses large memory
- 2) Depends on heuristic quality
- 3) Can be slow for large graphs
- 4) Heuristic design is difficult

Problem Reduction -

- Techniques to break a complex problem into smaller sub problems
- Each sub problem is solved independently
- Final solution is obtained by combining sub solutions
- Why?
 - (i) Simplifies complex problems
 - (ii) Reduces computational cost
 - (iii) Improves clarity & structure
 - (iv) Helps in systematic reasoning
 - (v) Useful in planning & expert systems
- Implemented using AND-OR graphs

- Step by Step Process :

- (i) Identify the main goal
- (ii) Divide into sub goals
- (iii) Solve sub goals individually
- (iv) Combine results
- (v) Obtain final solution

AO* Algorithm -

- AO* (AND-OR Star) is a graph search algorithm
- Used for problems represented as AND-OR graphs
- Generalization of A* algorithm
- Finds an optimal solution sub graph
- Useful in problem decomposition
- Why?

- (i) Some problems require solving multiple sub problems together
- (ii) Handles AND nodes and OR nodes
- (iii) Suitable where decisions are not linear paths

- AND & OR Nodes :

- (i) AND:
 - All children must be solved
 - eg. Task needs step A & step B
- (ii) OR:
 - Only one child needs to be solved
 - eg. choose route A or route B

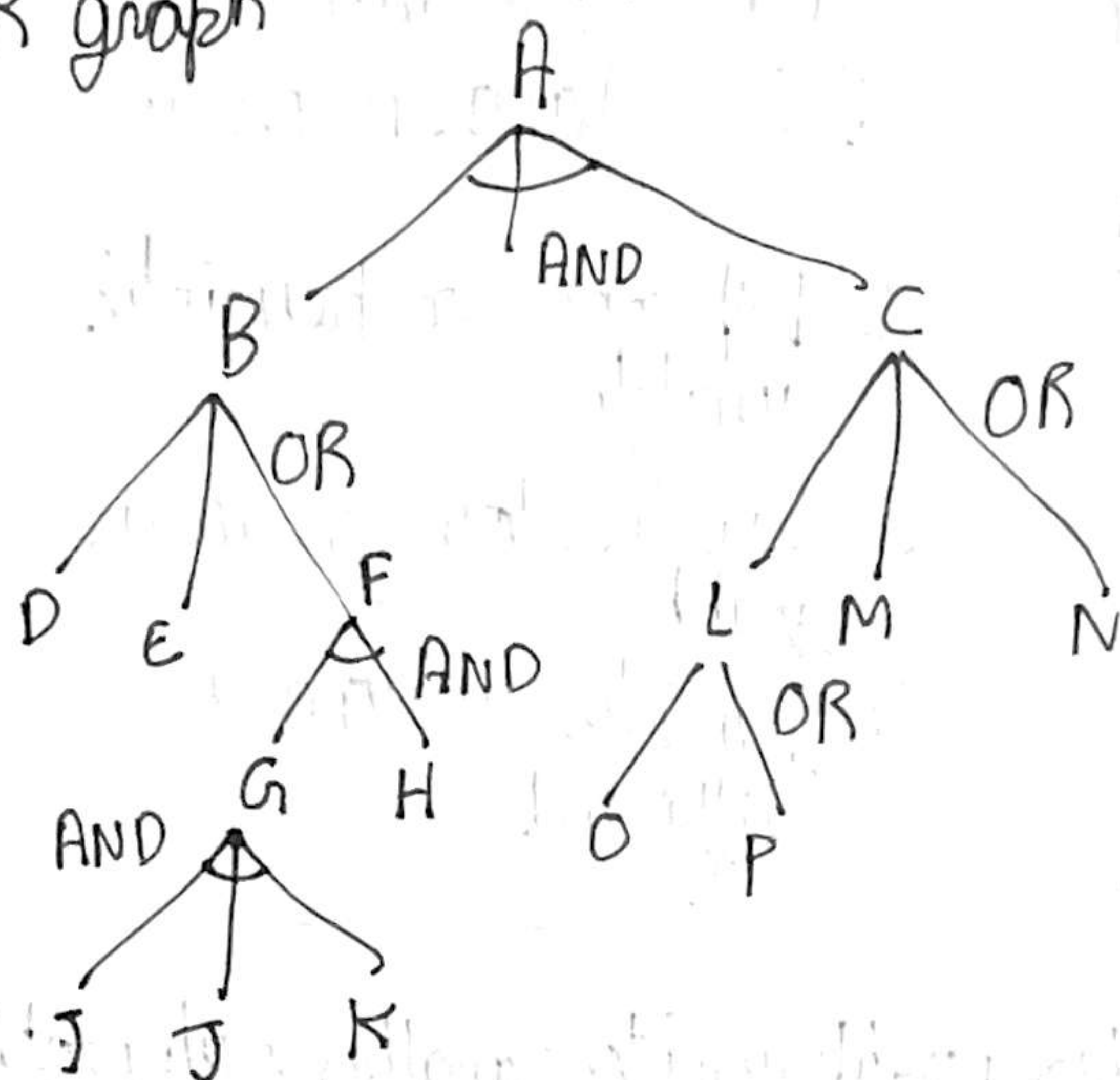
- Applications :

- (i) Planning systems
- (ii) Problem reduction tasks
- (iii) Game playing with complex goals
- (iv)
- (v) Diagnosis systems

- Cost Function (AO*):

- Uses cost like A*
$$f(n) = g(n) + h(n)$$
- Chooses the least cost solution graph

eg-AND/OR graph



#Constraint Satisfaction Problem (CSP) —

- A problem where values are assigned to variables
- Assignments must satisfy all given constraints
- A solution is valid only if no constraint is violated

- Key components:

- Variables = items to assign values to
- Domain = possible values for each variable
- Constraints = rules restricting assignments

- eg-

(i) Map Coloring:

- Variables = regions
- Domain = colors
- Constraint = adjacent regions $\neq$ same colour

(ii) Sudoku:

- Variables: empty cells
- Domain = {1-9}
- Constraint = no repetition in row, column or grid

- In AI -

- Solved using systematic search
- Uses pruning techniques to reduce search space

- Techniques:

- (i) Backtracking search = try values one by one $\rightarrow$ go back if conflict
- (ii) Forward checking = remove invalid values from future variables early
- (iii) Arc consistency = ensure that every value in variable has valid support
- (iv) Constraint Propagation = spread restrictions across all variables

Game playing in AI -

- technique to make decisions in competitive environments
- goal is to maximize gain & minimize opponent's chances
- Used in:
 - chess, ludo, poker
 - online strategy games
 - real time simulations
- ~~Game~~ cornerstone in AI
- AI is becoming an essential player in various gaming domains, reshaping how we experience & interact with ~~experience~~ games
- Refers to development of algorithms and models that enable machine to play & excel games that requires decision making, strategy, and problem solving
- Don't require much knowledge; only need the rules, legal moves & conditions of winning or losing the game
- Key characteristics:
 - (i) Multi agent environment (>1 player)
 - (ii) Conflicting objectives (one player's gain = another player's loss)
 - (iii) Turn based decision making
 - (iv) Uncertain outcomes

Adversarial Search -

- Deals with decision making when 2 or more agents have opposite goals

- Types of games:

Property	Example
Deterministic	Chess
Stochastic	Ludo
Perfect Info	Chess
Imperfect Info	Poker

Types of games

- (i) Single player games: Puzzles or solitaire
- (ii) Multiplayer games: Chess or Go
- (iii) Deterministic games (games with no randomness) : Chess
- (iv) Stochastic games (involving randomness) : Card games & dice games

- # Perfect vs Imperfect information games:
- (i) Perfect information = allows all players to access complete information about the game state at any point
eg - Chess and Go
 - (ii) Imperfect information = limit access to certain information, such as the opponent's hand in card game
eg - Poker

Zero-Sum games:
These games involve direct competition, where one player's gain is another player's loss.
eg - Chess, competitive multiplayer games

Role of Search algorithms in Game playing -

(i) Min-max algorithm -

- Widely used in 2 player, zero sum games like Chess. It evaluates all possible moves for both players by assuming the opponent will play optimally, minimizing the player's loss while maximizing gains

(ii) Alpha-Beta pruning -

- Optimization of minimax
- Reduces the no. of nodes evaluated, improving efficiency
- Eliminates branches in the search tree, that do not affect the final decision

(iii) Monte Carlo Tree Search (MCTS) -

- Combines random simulations with statistical analysis to determine the best moves.
- Particularly useful in games like Go and complex video games where state space is vast & exhaustive search is ~~meaningful~~ impractical

Min-max search algorithm -

- Backtracking algorithm used in decision making & game theory

- Helps choose the best possible move by assuming that the opponent also plays optimally

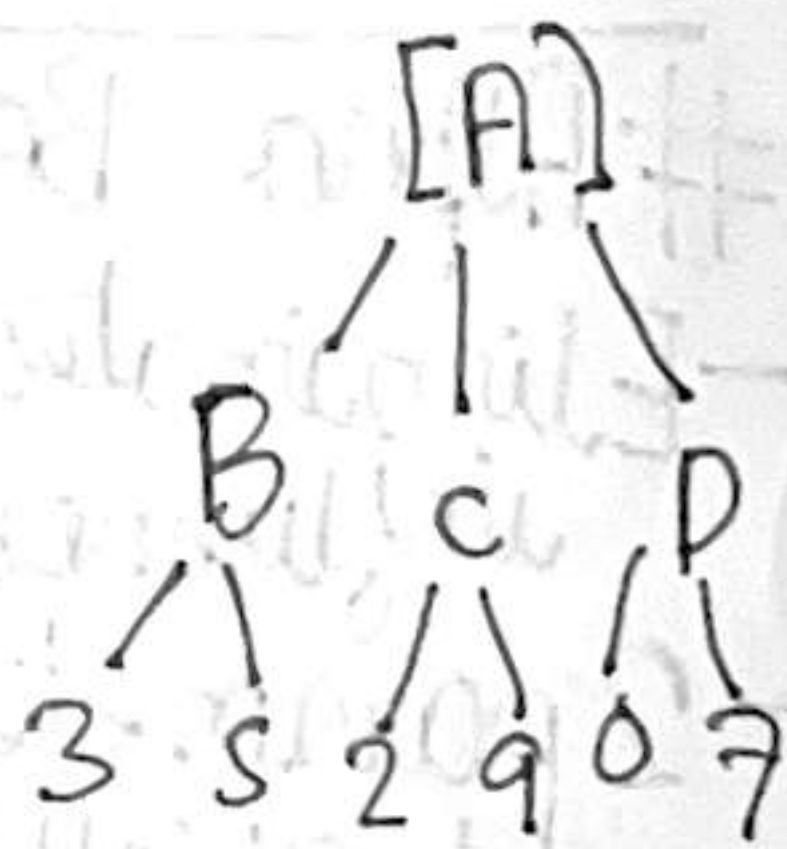
- MAX player → tries to maximize the score (AI's turn)

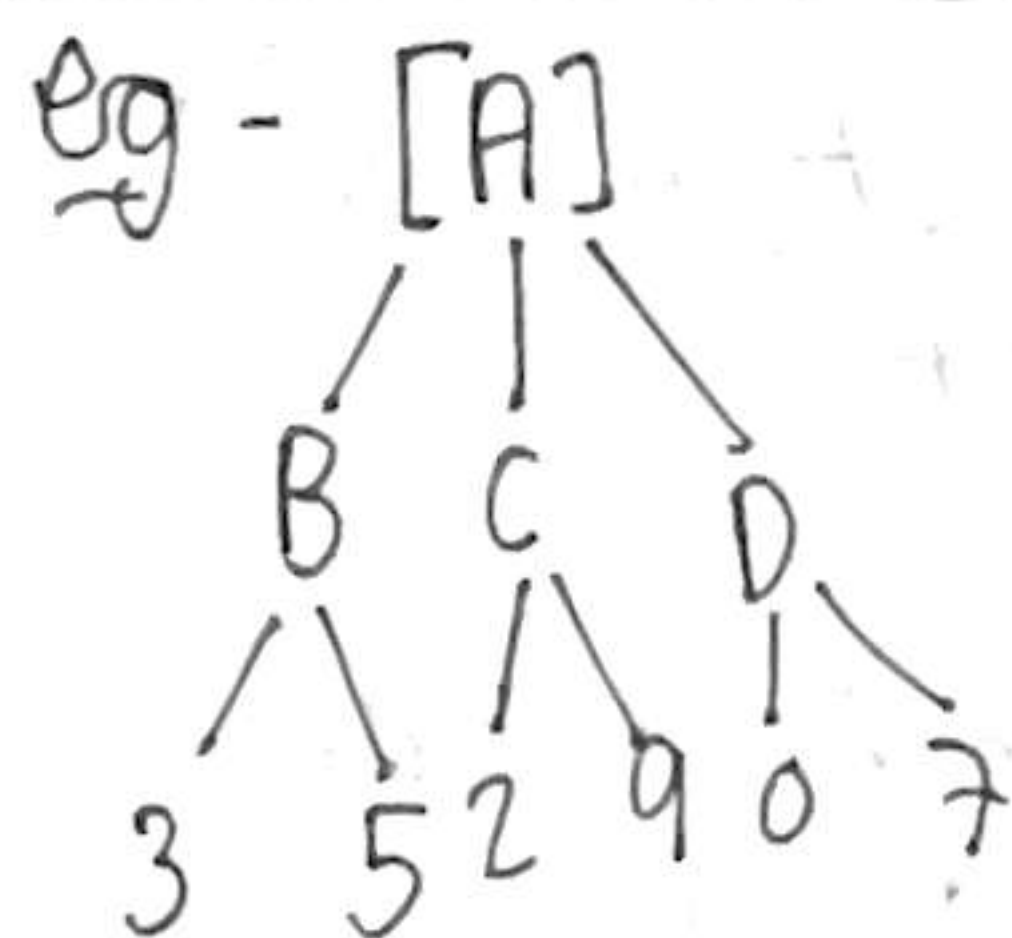
- MIN player → tries to minimize the score (Opponent's turn)

- eg - A = root node (AI's turn → MAX)

- B, C, D = opponent's moves (MIN nodes)

Leaf nodes = scores





1) Evaluate leaf nodes
Leaves already have scores:
3, 5, 2, 9, 0, 7

2) Propagate upward (MIN's turn)

Node B $\rightarrow \min(3, 5) = 3$

Node C $\rightarrow \min(2, 9) = 2$

Node D $\rightarrow \min(0, 7) = 0$

3) AI's turn (MAX)

Now MAX chooses the highest among $\{3, 2, 0\}$

$\max(3, 2, 0) = 3$

Best move: choose branch B

Step	Player	Action
Leaf	Values	3, 5, 2, 9, 0, 7
Level-1	Min	Takes 3, 2, 0
Level 2	Max	Chooses 3

- Used in 2 player, zero sum games where one player's gain is the other's loss (eg chess, tic tac toe)

- Idea is to maximize the AI's minimum gain (hence "Min-Max")

- Max player tries to maximize the score
- Min player tries to minimize the score

- How it works:

(i) Game states form a tree where:

- Node at max's turn choose the max value from child nodes
- Nodes at min's turn choose the min value from child nodes

(ii) Leaves of the tree have evaluated scores (utility & values)

(iii) The algorithm backtracks from leaves to root selecting the best move.

Alpha Beta Pruning -

- Eliminates branches in the game tree that cannot possibly influence the final decision

- 2 parameters are used:

- α (alpha): the best (max) value that the MAX player currently can guarantee; Lower Bound (Initially $-\infty$)

- β (beta): the best (min) value that the MIN player currently can guarantee; Upper Bound (Initially $+\infty$)

- Pruning Rule:
 $\rightarrow$ If at any point $\alpha \geq \beta$, we stop exploring that branch
 (no need to evaluate further)

- Process:

(i) Start from root
 - at root node A (MAX): $\alpha = -\infty, \beta = +\infty$

(ii) Explore left child (B)

- B is a MIN node: $\alpha = -\infty, \beta = +\infty$

- Evaluate its children (3, 5):

First child: 3 $\rightarrow \beta = \min(+\infty, 3) = 3$

Second child: 5 $\rightarrow \beta = \min(3, 5) = 3$

- Return value to parent (A): 3

- Now, α at A = $\max(-\infty, 3) = 3$

(iii) Explore middle child

- C is MIN node: $\alpha = 3$ (from A), $\beta = +\infty$

- Evaluate its children (6, 9):

First child: 6 $\rightarrow \beta = \min(+\infty, 6) = 6$

- Now, check pruning condition:

$\rightarrow$ since $\alpha(3) < \beta(6)$, continue

Second child: 9 $\rightarrow \beta = \min(6, 9) = 6$

- Return value to A: 6

- Now, $\alpha = \max(3, 6) = 6$

(iv) Explore right child

- D is MIN node: $\alpha = 6, \beta = +\infty$

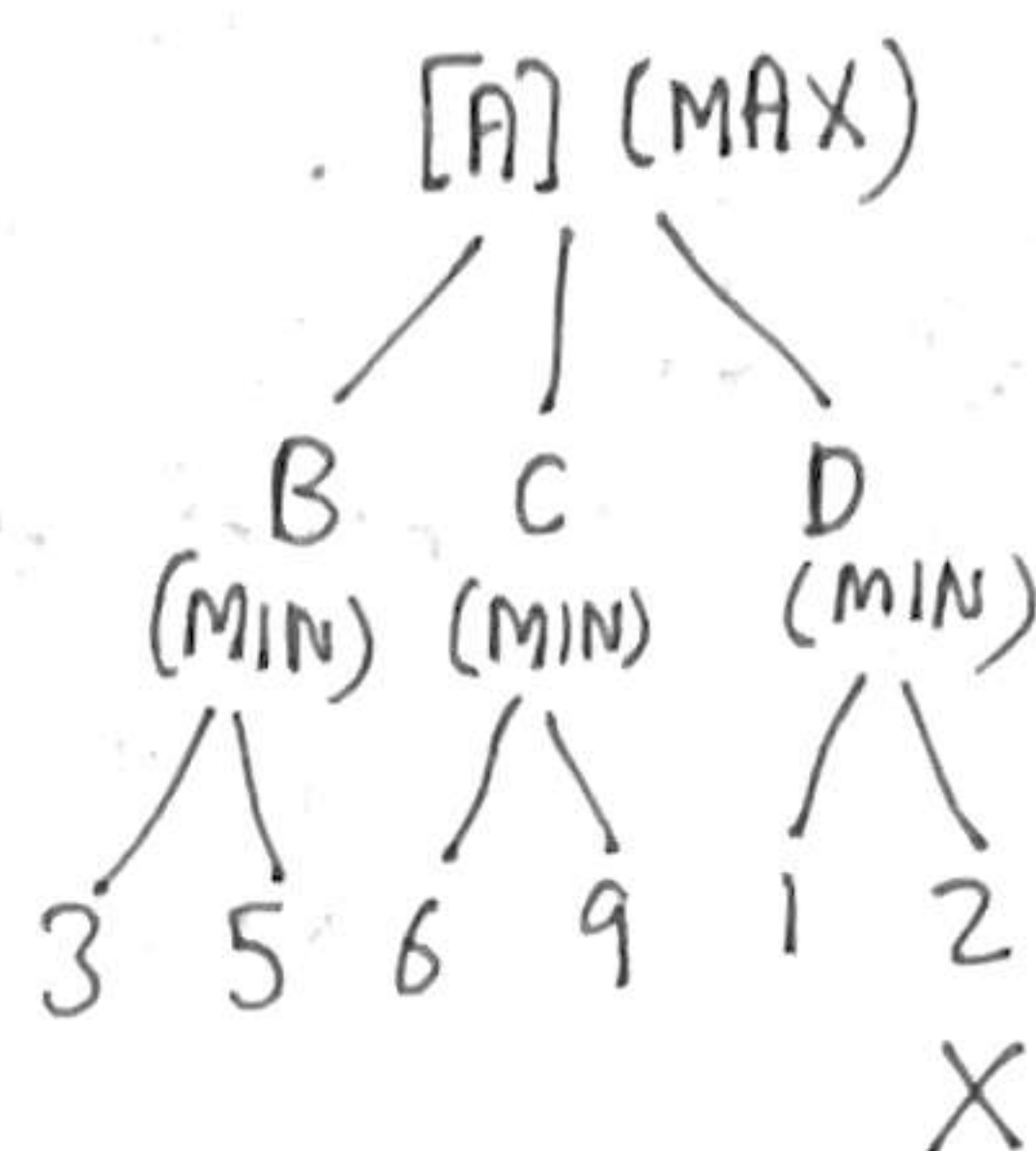
- Evaluate its children (1, 2):

First child: 1 $\rightarrow \beta = \min(+\infty, 1) = 1$

- Now $\alpha \geq \beta \rightarrow (6 \geq 1)$

- Prune the remaining child (2), no need to check

- Return value to A: 1



(v) Final decision at A
 MAX chooses among
 $\{3, 6, 1\}$

BEST move = 6 (from C)

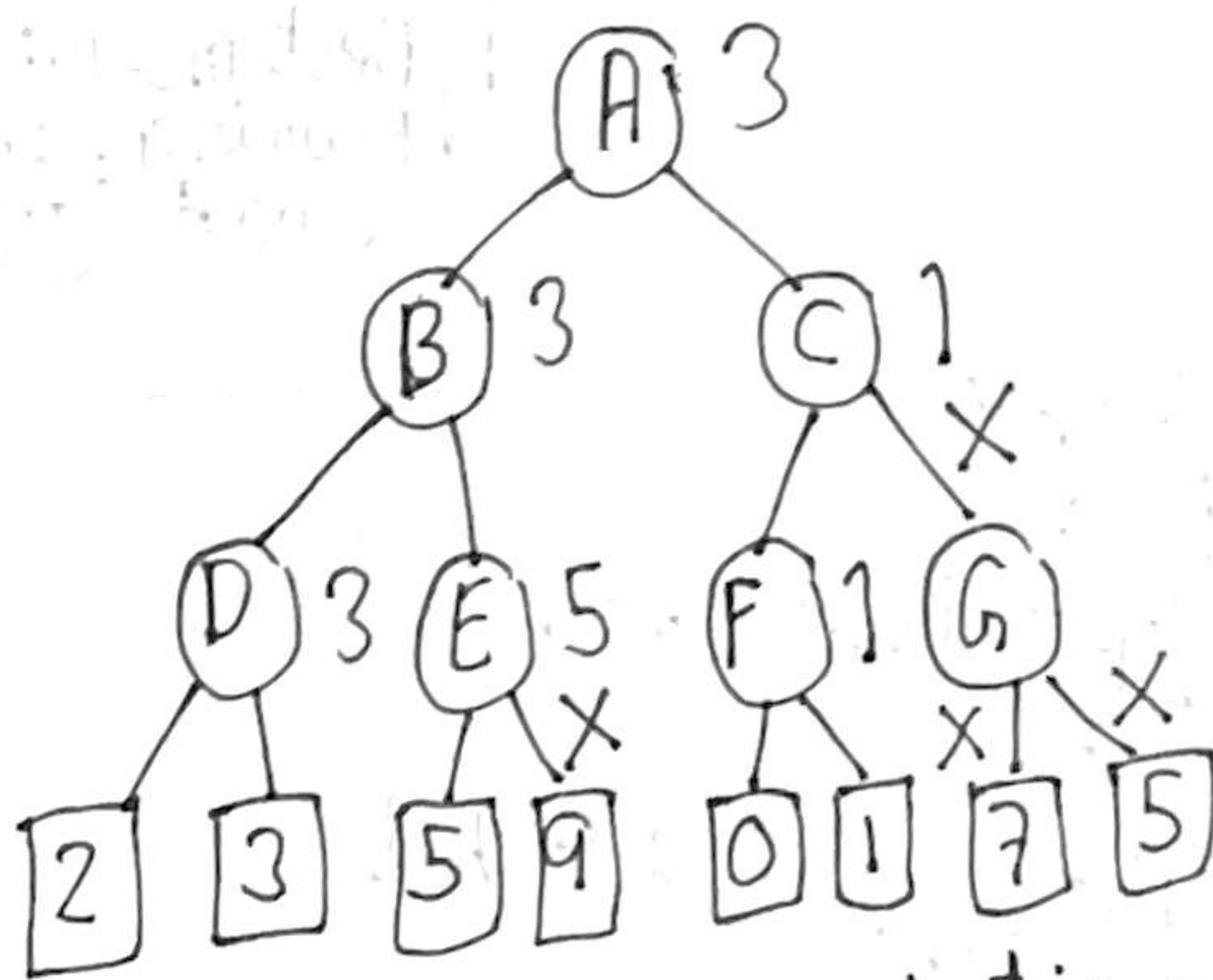
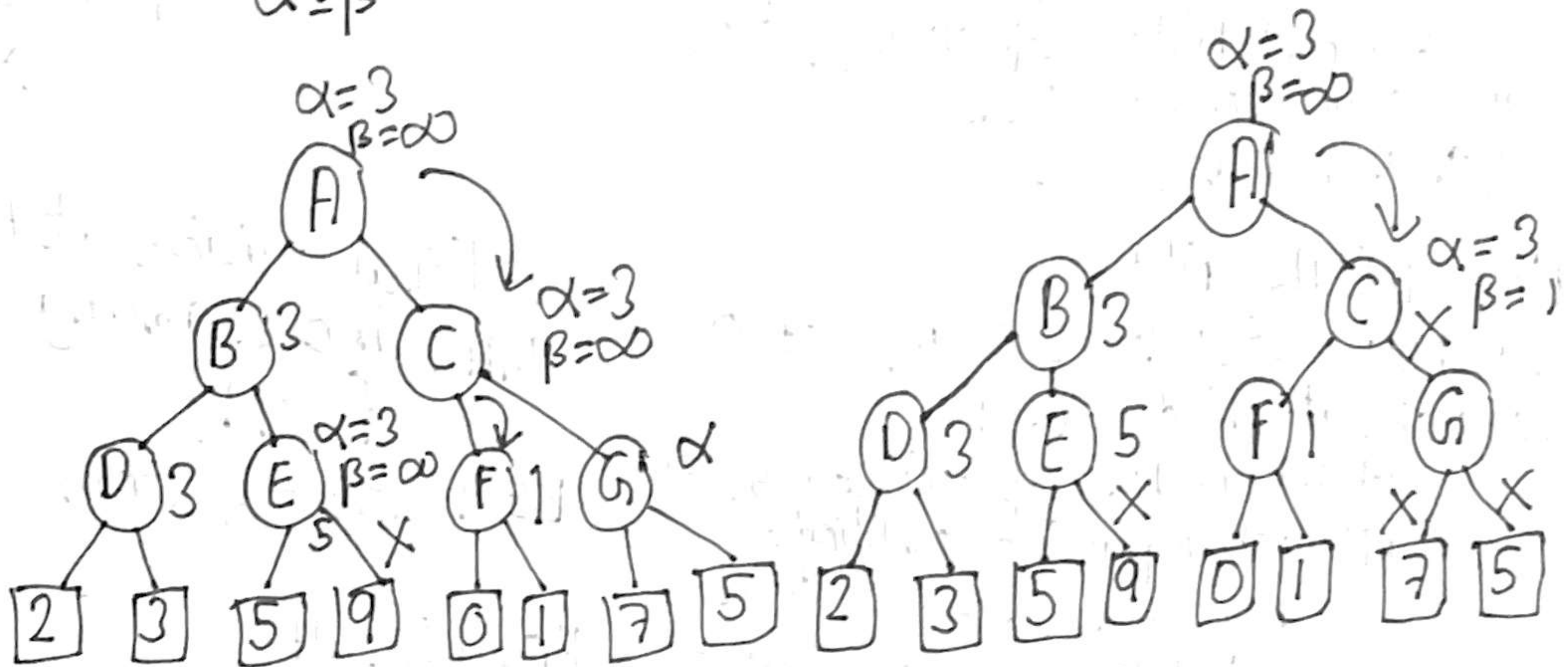
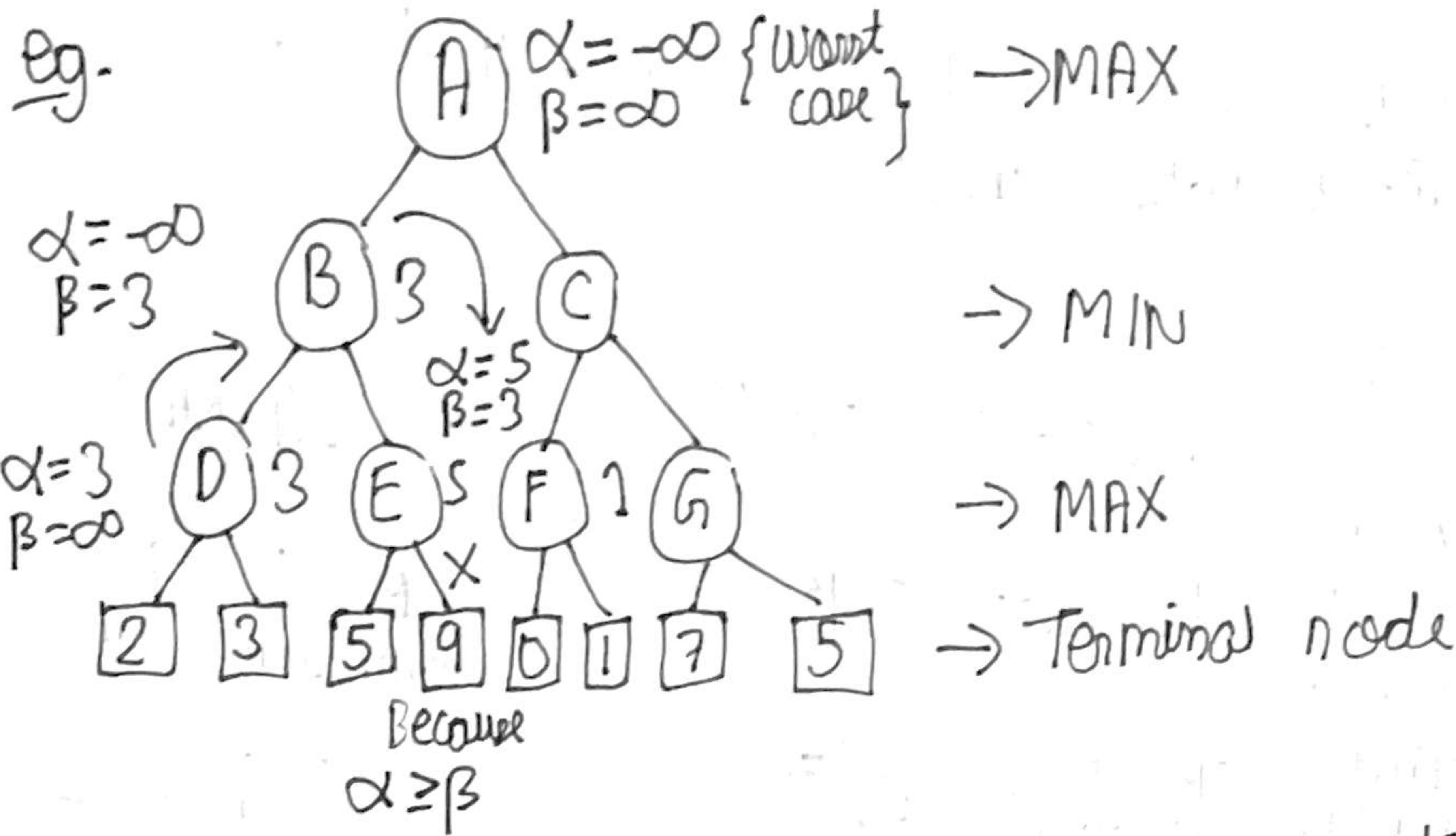
Final Result:

Best value: 6

Best move: Path A $\rightarrow$ C $\rightarrow$ 6

Pruning: one branch
 saved (value 2) was
 skipped

- If a branch cannot improve α or β values, it is pruned.
 - Search optimization technique that improves performance of minimax algorithm



- Key Benefits:

- (i) Improved efficiency
- (ii) Reduced time complexity
- (iii) Scalability

- Limitations:

- (i) Effective Pruning
- (ii) Limited Pruning efficiency
- (iii) Minimal impact
- (iv) Ineffective Pruning